

TÀI LIỆU KIẾN TRÚC HỆ THỐNG

(SYSTEM ARCHITECTURE DOCUMENTATION)

THÔNG TIN DỰ ÁN & TÀI LIỆU:

TÊN DỰ ÁN:	LIBIF — Hệ thống Số hóa Thư viện Thông minh (Library Digitization System)
PHIÊN BẢN:	v1.0 (Tài liệu Thiết kế Kiến trúc Hệ thống)
NGÀY LẬP:	Ngày 10 tháng 07 năm 2026
TÁC GIẢ:	Kiến trúc sư trưởng - Đội ngũ dự án LIBIF

1. LOẠI ỨNG DỤNG (APP TYPE)

Hệ thống LIBIF là một ứng dụng Web Doanh nghiệp (Enterprise Web Application) tích hợp, phục vụ hai luồng nghiệp vụ tương tác khác biệt:

- Cổng Đọc giả (Reader Portal): Hệ thống web hướng ngoại (Responsive Web), hỗ trợ đọc giả tra cứu và đọc sách trực tuyến bảo mật.
- Cổng Quản trị & Nghiệp vụ (Librarian Admin Panel): Hệ thống web hướng nội để thủ thư tải tệp tin PDF sách đã quét, kiểm định, nhập metadata và xem báo cáo hoạt động.

2. ĐÁNH GIÁ VÀ LỰA CHỌN KIẾN TRÚC TỔNG THỂ

Để xây dựng nền tảng vững chắc cho hệ thống, nhóm kiến trúc tiến hành phân tích so sánh hai mô hình kiến trúc tổng thể phổ biến: Monolith và Microservices:

Phương án xem xét	Ưu điểm cốt lõi	Nhược điểm / Thách thức
Modular Monolith (Đơn khối mô-đun)	- Triển khai đơn giản, chi phí vận hành thấp. - Gọi hàm trực tiếp, hiệu năng cao. - Các mô-đun cô lập rõ ràng ở mức mã nguồn.	- Toàn bộ hệ thống dùng chung cơ sở dữ liệu vật lý. - Chia sẻ chung tài nguyên CPU/Memory của một máy chủ.
Microservices (Kiến trúc vi dịch vụ)	- Cho phép mở rộng độc lập từng phân hệ. - Tách biệt cơ sở dữ liệu vật lý. - Linh hoạt về mặt công nghệ.	- Hạ tầng phức tạp, tốn kém chi phí vận hành. - Trễ mạng khi giao tiếp RPC/REST giữa các dịch vụ. - Rất khó để triển khai cho đội ngũ nhỏ.

Quyết định & Lập luận lựa chọn: Modular Monolith

Hệ thống quyết định áp dụng kiến trúc Modular Monolith. Giải pháp này giúp tối giản hóa quy trình vận hành và triển khai trong giai đoạn MVP, tiết kiệm tài nguyên hệ thống và đẩy nhanh tốc độ đưa sản phẩm ra thực tế. Thiết kế tách biệt các thư mục nghiệp vụ rõ ràng (Authentication, Upload, Catalog, Reader, Processing) đảm bảo code luôn sạch sẽ và sẵn sàng bóc tách riêng mô-đun OCR thành một Microservice độc lập sau này nếu gặp bài toán quá tải tài nguyên.

3. ĐÁNH GIÁ LỰA CHỌN MÔ HÌNH XỬ LÝ DỮ LIỆU (PDF & OCR)

Quy trình xử lý file PDF thô sau khi quét (Kiểm định → Nén tối ưu → Chạy OCR tạo searchable PDF → Trích xuất chỉ mục tìm kiếm) đòi hỏi một mô hình xử lý luồng dữ liệu mạnh mẽ. Chúng tôi đánh giá hai mô hình chính:

Mô hình xem xét	Đặc tính & Ưu điểm	Độ phù hợp với xử lý sách số
Pipe and Filter (Đường ống & Bộ lọc)	- Chia nhỏ quy trình thành các bộ lọc độc lập. - Dữ liệu di chuyển liên tục qua các bộ lọc.	- Cực kỳ phù hợp cho luồng xử lý file PDF thô. - Dễ dàng kiểm định chất lượng ở từng công đoạn.
Batch Sequential (Xử lý lô tuần tự)	- Xử lý toàn bộ lô dữ liệu lớn trước khi sang bước tiếp theo. - Mỗi bước phải đợi bước trước hoàn thành 100%.	- Gây độ trễ (latency) rất cao đối với file sách dày. - Dễ gây tắc nghẽn hoặc sập bộ nhớ nếu file quá lớn.

Quyết định: Pipe and Filter kết hợp Hàng đợi tác vụ

Hệ thống lựa chọn kiến trúc Pipe and Filter. Mỗi file PDF khi tải lên sẽ đi qua một "đường ống" xử lý gồm 4 bộ lọc. Để hiện thực hóa thiết kế này và tránh tắc nghẽn tài nguyên, chúng tôi sử dụng Redis + BullMQ làm công cụ điều phối hàng đợi tác vụ bất đồng bộ. Mỗi bộ lọc là một worker độc lập, giúp hệ thống hoạt động ổn định và tối ưu tài nguyên CPU.

[1. Validation] → [2. Compression] → [3. OCR Text] → [4. Indexing]
(Kiểm tra file) (Nén file) (Tạo chữ n) (Đánh chỉ mục)

4. ĐÁNH GIÁ LỰA CHỌN MÔ HÌNH GIAO TIẾP VÀ TƯƠNG TÁC

Đối với việc truyền đạt thông tin giữa các thành phần nghiệp vụ (Ví dụ: thông báo có file mới được upload, thông báo OCR đã xong, cập nhật chỉ mục tìm kiếm), chúng tôi so sánh hai mô hình giao tiếp:

Mô hình giao tiếp	Mô tả & Điểm mạnh	Thử thách / Khó khăn
Request - Response (Giao tiếp đồng bộ)	- Phản hồi tức thời cho người dùng. - Quy trình lập trình đơn giản, trực quan (REST API).	- Gây nghẽn luồng xử lý (Blocking) nếu tác vụ kéo dài. - Khó khăn khi tích hợp nhiều tác vụ nền.
Event-Driven (Hướng sự kiện)	- Ghép nối lỏng (Loose coupling) giữa các mô-đun. - Tách biệt thời gian xử lý các tác vụ nền.	- Đòi hỏi thiết kế phức tạp (Event Bus, Event Handler). - Khó theo dõi luồng dữ liệu (Data flow) trực quan.

Quyết định: Mô hình Lai (Hybrid Communication Model)

Hệ thống lựa chọn mô hình Lai (Hybrid):

- Sử dụng Request-Response (REST API qua HTTP) cho các tương tác đồng bộ trực tiếp của độc giả (đăng nhập, tìm kiếm, đọc trực tuyến).
- Áp dụng kiến trúc Event-Driven (Hướng sự kiện bất đồng bộ) cho các nghiệp vụ xử lý nền.

Khi thủ thư tải file sách lên thành công, hệ thống phát ra sự kiện `BookUploadedEvent` và trả về phản hồi tức thời. Tác vụ chạy Pipeline OCR sẽ được kích hoạt bởi Event Handler để xử lý dưới nền mà không chặn kết nối.

Uploader Module → BookUploadedEvent → Event Bus (EventEmitter) → OCR Worker
(Kích hoạt tác vụ)

5. CẤU TRÚC THÀNH PHẦN (SYSTEM COMPONENTS)

Hệ thống số hóa được chia tách thành các phân hệ nghiệp vụ độc lập, đóng gói gọn gàng bên trong cấu trúc Modular Monolith để dễ quản lý phát triển:

- Mô-đun Đăng nhập (Auth): Quản lý phân quyền, đăng ký, đăng nhập tài khoản của thủ thư, độc giả và quản trị viên.
- Mô-đun Đăng tải (Upload): Nhận file PDF thô từ thủ thư, lưu trữ vào kho lưu trữ đối tượng và kích hoạt sự kiện xử lý.
- Mô-đun Tra cứu (Catalog): Lập chỉ mục thông tin sách, hỗ trợ tìm kiếm nâng cao và tra cứu mã ISBN.
- Mô-đun Trình đọc (Reader): Xác thực quyền truy cập sách của độc giả, tạo presigned URLs bảo mật và render sách trực tuyến.
- Mô-đun Xử lý (Processing): Điều phối quy trình Pipe and Filter chạy nén file và OCR text bất đồng bộ.

5.2 Chi tiết Tech Stack lựa chọn

Công nghệ được chọn lựa kỹ lưỡng để hỗ trợ tối ưu cho các mô hình kiến trúc đã được chọn lọc:

Phân lớp	Công nghệ lựa chọn	Vai trò & Lý do lựa chọn
Frontend	Next.js (React)	Tạo giao diện Portal đọc sách responsive mượt mà và Server-Side Rendering giúp tải trang nhanh chóng.
Backend	NestJS (Node.js)	Hỗ trợ Modular Architecture tốt nhất cho Node.js, có sẵn hệ thống EventEmitter nội bộ giúp lập trình hướng sự kiện.
Database	PostgreSQL	Lưu trữ thông tin quan hệ metadata sách. Sử dụng `pg_trgm` để thực hiện chỉ mục tìm kiếm Full-text search.
File Storage	AWS S3 hoặc MinIO	Kho lưu trữ đối tượng tiêu chuẩn chuyên dùng để lưu trữ file PDF vật lý.
Queue/Broker	Redis + BullMQ	Hàng đợi tin nhắn chịu tải cao, điều phối luồng xử lý Pipe & Filter OCR bất đồng bộ.
OCR Engine	Tesseract OCR	Công cụ OCR nguồn mở nhận diện chữ tiếng Việt chính xác và miễn phí hoàn toàn.

6. AN TOÀN DỮ LIỆU & THUỘC TÍNH CHẤT LƯỢNG KIẾN TRÚC

6.1 Cơ chế bảo mật và chống tải file PDF

Để bảo vệ bản quyền sách của thư viện, hệ thống thiết kế cơ chế bảo mật tệp tin nghiêm ngặt:

- Presigned URLs: Các file PDF lưu trên MinIO/S3 không được mở công khai. Khi độc giả yêu cầu đọc sách, hệ thống tạo một liên kết tạm thời có mã hóa bảo mật chỉ có hiệu lực trong vòng 15 phút.
- Chống tải xuống trực tiếp: Trình đọc PDF trực tuyến (Reader) dựng trang dạng Canvas động thay vì nhúng thẻ `iframe` trực tiếp. Vô hiệu hóa tính năng bấm chuột phải, chặn phím F12 và vô hiệu hóa tính năng in tệp tin để giảm thiểu nguy cơ sao chép.

6.2 Thuộc tính chất lượng kiến trúc cốt lõi

- Khả năng bảo trì (Maintainability): Sự phân tách mô-đun rõ ràng trong cấu trúc Modular Monolith cho phép cập nhật logic của từng mô-đun độc lập. Ví dụ, ta có thể đổi thư viện OCR cục bộ sang API Cloud mà không cần sửa code giao diện.
- Khả năng mở rộng (Scalability): Thiết kế sẵn sàng bóc tách (Ready-to-Split). Việc sử dụng Redis + BullMQ làm trung gian giúp tách biệt hoàn toàn máy chủ xử lý file khỏi máy chủ web nghiệp vụ khi cần thiết.
- Tính chịu lỗi (Fault Tolerance): BullMQ hỗ trợ cấu hình thử lại tự động (Retry 3 lần) cho các tác vụ OCR thất bại do lỗi mạng hoặc file lỗi nhẹ, hạn chế tối đa việc tắc nghẽn luồng xử lý.

7. KẾT LUẬN KIẾN TRÚC (ARCHITECTURAL CONCLUSION)

Kiến trúc được đề xuất cho Hệ thống Số hóa Thư viện đã được chọn lọc kỹ lưỡng:

- Lựa chọn Modular Monolith để tối ưu hóa nhân sự và chi phí hạ tầng ban đầu.
- Áp dụng mẫu thiết kế Pipe and Filter dưới dạng các Background Workers kết hợp hàng đợi bất đồng bộ để giải quyết việc xử lý file nặng.
- Áp dụng cơ chế Hybrid Communication (REST và Event-Driven) để nâng cao độ phản hồi của ứng dụng.

Đây là bản thiết kế tối ưu, có độ bảo mật cao và sẵn sàng đáp ứng nhu cầu mở rộng lâu dài của thư viện số.